ZT Chromatogram-Feature Redesign

Two axes, computed at the right level: fixes broken features *and* the memory win

feat/sciex-zt-scanning

2026-07-09

1 The core insight

For ZT scanning DIA the per-precursor "chromatogram" (its weight/fragment trace across all its PSM rows) mixes **two orthogonal axes** that the current code treats as one:

- **Within-metascan (shape) axis** — across the $2k{+}1{=}13$ Q1 bins of *one* cycle, the weight and each fragment intensity follow the transmission **triangle**. This encodes "do the fragments ride the metascan shape?" — discriminative (real precursors ride the triangle; interference does not).
- **Across-metascan (RT / elution) axis** — one point per cycle: "do the fragments co-elute over the gradient?" The classic chromatogram feature.

Today both are computed on the raw **13-per-cycle jagged trace**, so:

1. **The fragment correlations are dominated by the shape axis** (13 within-cycle points per ∼n_cycles elution points), so `frag_corr_strength` etc. are really *shape* features that happen to score well — while the *elution* signal they were meant to capture is diluted.
2. **The MS1 correlations are broken** (see §2).
3. `n_scans` **is an artifact** ($\approx 13\times$ the true peak width).

The plan: compute each feature at its natural level — shape features from the within-cycle profile, elution features from the clean one-point-per-cycle trace. This corrects the features *and*, because everything is computed at the meta-scan level (from data the collapse already gathers, plus a per-cycle fragment profile), it is the memory/efficiency win we were chasing.

2 Evidence: the MS1 features are broken

`features.jl:1026--1038` (`_add_ms1_chromatogram_features!`):

```
1026 for k in 1:npts
1027     v_m0[k] = m0[i_orig]     # MS1 M0 intensity for this (precursor, scan) row
1028     v_w[k]  = w[i_orig]      # deconv weight for this bin
1029 end
1030 c_wm0 = _frag_pcor(v_w, v_m0)   # Pearson(weight trace, M0 trace)
```

There is **one MS1 scan per cycle**, so all 13 bins of a cycle read the *same* `m0` ⇒ `v_m0` is a step function (13 identical values/cycle), while `v_w` is the triangle (13 varying values/cycle). The Pearson is then corr(within-cycle triangle, within-cycle constant) — the within-cycle variation is pure noise against a flat line. This is why all four MS1-chromatogram features have **≈0 LGBM gain**. At the meta-scan level (one weight vs one M0 per cycle) it becomes corr(elution, elution) — the correlation it was meant to be.

3 Feature-by-feature redesign

Current LGBM gains (MainSearch per-file / 2nd-pass Pass-1) shown to prioritise.

Feature (gain MS / P1)	Axis today	Axis in redesign	Action
MS1-chromatogram (all ≈0 gain — broken)			
`ms1_corr_weight_m0` (0/0)	mixed	**elution**	recompute on clean per-cycle trace (fix)
`ms1_corr_m0_m1` (0/0)	mixed	**elution**	recompute on clean per-cycle trace (fix)
`ms1_apex_offset_irt` (0/0)	mixed	**elution**	apex on clean trace
`ms1_weight_apex_to_m0_apex_irt` (115/249)	mixed	**elution**	apex on clean trace
Fragment-chromatogram — split the axes			
`frag_corr_strength` (2871/1001)	shape-dom.	**split**	shape + elution
`frag_corr_effective_n` (1419/1586)	shape-dom.	**split**	shape + elution
`frag_apex_dispersion_irt` (827/**2249**)	mixed	**split**	shape (within-cycle apex spread) + elution (across-cycle)
`frag_corr_best_m0` (87/95)	shape-dom.	**split**	shape + elution
`n_correlated_fragments` (80/14)	shape-dom.	**split**	count over shape corr >thr + over elution corr >thr
`n_correlated_fragments_bitvec_and` (71/312)	shape-dom.	**split**	shape + elution (thresholded)
`delta_frame_peak_center` (226/230)	mixed	**elution**	peak-center drift over cycles
`n_scans` (2275/16)	raw count	**meta count**	= number of cycles (peak width)
New shape features (the discriminative part, now explicit)			
`frag_shape_corr` (new)	—	**shape**	mean over cycles of Pearson(frag profile, weight triangle)
`frag_shape_corr_apex` (new)	—	**shape**	same, at the apex cycle only

Guiding rule: **a correlation "across the 13 bins" is a shape feature; a correlation "across cycles" is an elution feature.** We currently only have the (mislabelled) former; we add the latter and fix MS1.

Per review (2026-07-09): split *all* of the fragment correlations into a within-metascan *shape* version and an across-cycle *elution* version — not just the top-gain three. So each of `frag_corr_strength`, `frag_corr_effective_n`, `frag_apex_dispersion_irt`, `frag_corr_best_m0`, `n_correlated_fragments`, `n_correlated_fragments_bitvec_rank` gets a `*_shape` and a `*_elution` column. Generate the full set first, then **whittle down by LGBM gain** after the first A/B. The elution versions **mirror the current (develop) computation** but at the meta-scan/cycle level.

4 Existing precedent: the flanking core already does this right

The codebase *already* computes a cycle-based contiguity/gap measure at the meta-scan level — we should mirror it rather than invent new machinery.

`_mainsearch_flanking_core_bounds!` (scoring.jl:799), called from `add_trace_and_fragment_features!(best_psms, psms)` **after the collapse**, sorts a precursor's rows by `cycle_idx` and walks outward from the apex, admitting a neighbour *only if it is the adjacent cycle*:

```
854 UInt32(cycle_idxs[previous]) == UInt32(cycle_idxs[current]) - UInt32(1) ||
        break  # walk left
855 ...
856 UInt32(cycle_idxs[next_row]) == UInt32(cycle_idxs[current]) + UInt32(1) ||
        break  # walk right
```

i.e. it stops at a cycle gap ("a large enough gap ⇒ not the same sequence"), keyed on **cycles, not raw scans**. Because its input is the collapsed meta-PSM table (one row per cycle), this is *already correct for ZT* and produces `n_contiguous_scans`.

Consequences for this plan:

- The broken features (`frag_corr_*`, `ms1_corr_*`, and the artifact `n_scans`) are exactly the ones in `add_chromatogram_features!`, which runs *pre*-collapse on the 13-per-cycle jagged trace. The redesign moves them into a post-collapse pass that reuses the same cycle-contiguity idea for the *elution* gap/adjacency.
- `n_scans` (pre-collapse, ≈ 13×cycles, an artifact — yet gain 2275) vs `n_contiguous_scans` (post-collapse, real cycle count) is an existing inconsistency; the redesign's `n_scans` = cycle count aligns them.
- The elution `frag_corr` should build each fragment's per-cycle trace over the *contiguous cycle core* (or gap-aware), not a naive correlation over all rows — matching the flanking core's notion of "the same eluting peak".

5 What the collapse must carry

`collapse_to_metascans` already gathers the 13 weights/cycle into `metascan_w_*` (metascan_collapse.jl:113). Extend it to also gather, per (precursor, cycle):

- the **per-fragment profile**: for each of the 8 fragments, its intensity in each of the 13 bins (from `frag1_int..frag8_int` on the raw rows) — used for the *shape* correlations and to form each fragment's per-cycle intensity;
- the **per-cycle combined values** for the *elution* trace: one weight and one intensity per fragment per cycle (triangle-weighted sum, matching the integration collapse), plus the cycle's M0/M1 (already per-cycle).

Two economical options:

1. **Compute shape features inside the collapse** (it already has the 13 weights; add the 13×8 fragment gather) and emit *scalars* (`frag_shape_corr`, `frag_shape_corr_apex`) + the per-cycle combined fragment intensities. Smallest footprint; no 13×8 columns stored.

2. Store the raw per-cycle fragment profiles as columns and compute everything in a ZT chromatogram pass. More flexible, more columns.

Option 1 is preferred (keeps the meta-PSM table narrow).

6 Implementation

1. `metascan_collapse.jl` — extend the per-(precursor,cycle) gather to also read `frag1..8_int`; compute `frag_shape_corr` / `frag_shape_corr_apex`; emit per-cycle combined fragment intensities (triangle-weighted) as 8 columns (or a packed form) on each meta-PSM. Keep `metascan_w_*` + the existing `ZT_PROFILE_FEATURES`.

2. **New** `_add_zt_elution_features!` (features.jl, ZT-gated) — replaces `_add_ms1_chromatogram_features!` + `_add_fragment_chromatogram_features!` for ZT. Operates on the collapsed meta-PSMs (one point/cycle): builds each precursor's clean per-cycle traces (weight, M0, M1, per-fragment combined intensity) and computes the *elution* versions of the features above + `n_scans` = n_cycles. Non-ZT keeps the current functions unchanged.

3. `MainSearch.jl process_search_results!` — for ZT, collapse first (as in the reverted reorder), then call `_add_zt_elution_features!` on the 3.2M meta-PSMs; `prepare_psm_features!` runs there too (byte-identical). Drop the 35M `permute` (collapse re-sorts).

4. **Feature lists** — add the new shape/elution columns to `PRESCORE_FEATURES` and the 2nd-pass config (`model_config.jl`); the LGBM `hasproperty` filter means non-ZT simply won't have them.

7 Verification & rollout

This is a feature *change*, not byte-identical — the goal is *better or equal* IDs at lower cost. Use the harness we built:

- 3-file A/B vs `results_3rep_iso00` baseline (73,657 prec / 12,228 PG / CV 8.48% / 6.47 min). Success = IDs ≥ baseline, CV ≤ baseline, and **candidate count into 2nd-pass ≤ baseline** (the naive meta reduction inflated it +36%; the shape features should prevent that).

- Inspect the new features' LGBM gains — expect the elution MS1 corr to go from ≈0 to non-trivial, and the shape corr to retain the discriminative power.

- Env/ZT-gated (`PIONEER_ZT_METASCAN_K`); non-ZT path byte-identical. Land the collapse-first + `prepare` move (byte-identical part) first, then the feature functions.

Efficiency: all new features compute at the meta-scan level (3.2M) or inside the collapse (from data it already touches), so we get the memory reduction (∼−17% Main Search alloc, −7 GB peak RSS from the reorder measurement) *without* the candidate inflation — because we no longer discard the shape axis.

8 Decisions from review (2026-07-09)

1. **Shape corr (fragment-vs-weight within a metascan): KEEP.** Expected to add information beyond `zt_tri_cosine` (which is weight-vs-triangle only). This is fragment-vs-weight, per fragment. *Follow-up:* once implemented, also try fragment-vs-*triangle* variants and compare.

2. **Elution intensity per cycle: implement BOTH and test.** Provide the triangle-weighted combined intensity *and* the center-bin intensity as an env-gated toggle; A/B them (no strong prior which wins).

3. `delta_frame_peak_center` **+ elution features: mirror develop at the cycle level.** Post-collapse our cycle notion is accurate, so keep develop's computation but drive it off the one-point-per-cycle trace / contiguous cycle core. Don't redesign the math, just re-level it.

4. **Split scope: split ALL fragment correlations** into `*_shape` + `*_elution` initially (six pairs; see §3), generate the full set, then **whittle down by LGBM gain** after the first A/B. Feature-count ballooning is fine as a first pass since the `hasproperty` filter + gains will prune.

Resulting first-pass feature set (ZT).

- MS1 (4): elution recompute of `ms1_corr_weight_m0`, `ms1_corr_m0_m1`, `ms1_apex_offset_irt`, `ms1_weight_apex_to_m`
- Fragment ($12 = 6$ pairs): {`frag_corr_strength`, `frag_corr_effective_n`, `frag_apex_dispersion_irt`, `frag_corr_best_m0`, `n_correlated_fragments`, `n_correlated_fragments_bitvec_rank`} × {`_shape`, `_elution`}.
- `n_scans` = cycle count; `delta_frame_peak_center` (elution, cycle-leveled).
- Elution intensity toggle: `PIONEER_ZT_ELUTION_INTENSITY` ∈ {triangle, center}.

Then A/B (IDs ≥, CV ≤, candidates ≤ baseline), read the gains, and prune.